DEPI PROJECT

NUVANTA

AI-Powered Enterprise Financial & Operations Management System

Project Planning & Management

Document	Project Planning & Management
Project	NUVANTA
Instructor	Ahmed Mohamed Maher
Track	Full-Stack .Net Web
Team	Menna Atef, Nour Hany, Weam Emad, Nourhan Fawzy, Omar Mohamed

CONTENTS

1. Project Proposal
2. Task Assignment & Roles
3. Risk Assessment & Mitigation Plan
4. KPIs (Key Performance Indicators)

1. Project Proposal

1.1 Overview

NUVANTA is a single web platform that connects a company's money with its operations. The company sets a total budget, distributes it across departments and cost centers, and every purchase, expense, salary or sale is reflected automatically in the budget, the accounting ledger, the inventory and the analytics. Each user sees and does only what their role allows, and an AI layer helps management make better decisions.

NUVANTA is not just a wallet, an accounting dashboard, or an inventory system. It is an integrated system that starts at the budget and ends with real execution and result analysis.

1.2 Problem Statement

In many companies, budgets, purchasing, payments, inventory and accounting live in separate tools or spreadsheets. This causes:

- Budgets that are exceeded without anyone noticing in time.
- Purchases approved without checking available budget or cash.
- No clear difference between planned money, committed money, spent money and cash actually in the bank.
- Weak audit trails (who changed what, and when).
- Late, manual decisions because analysis and forecasting are not built in.

1.3 Objectives

1. Build one platform that links Budget, Procurement, Payment, Accounting, Inventory and Analytics.
2. Enforce role-based access (RBAC) and data scope in the backend on every request, not only in the UI.
3. Support Maker–Checker (segregation of duties): the person who creates a financial transaction does not approve it.
4. Make financial integrity a guarantee: atomic transactions, no duplicate payments, a full audit log, and no silent edits to posted entries.
5. Provide explainable AI: Cash-Flow & Budget Forecasting, Anomaly Detection & Financial Risk, and an AI Business Assistant that respects the user's permissions.
6. Deliver a working end-to-end demo: create a budget, submit a purchase request, approve it, record receipt, record payment, then see the effect on budget, accounting and inventory.

1.4 Scope

In scope – Phase 1 (Core MVP)

- Users, Roles, Permissions, Departments.
- Budget setup and allocation (Total Budget, Departments, Cost Centers, Categories).
- Purchase Requests and Approval Workflow.
- Suppliers, Purchase Orders, Goods Receipt, Supplier Invoices.
- Payments: sandbox simulation and recording of external transfers (with reference and receipt).
- Cash transactions and budget tracking (Planned, Committed, Spent, Available).
- Basic accounting ledger and basic inventory.
- Dashboards by role, notifications, approval inbox, and audit logs.
- Tests for permissions and financial transactions.

Selected AI features

- Cash-Flow & Budget Forecasting.
- Anomaly Detection & Financial Risk (rules first, then a model such as Isolation Forest).
- AI Business Assistant (natural-language questions over data the user is allowed to see).

Planned for later phases

- Phase 2: Sales & Customers, HR & Payroll, Production & Costing, wider financial reports, bank reconciliation, real payment integration if officially available.
- Phase 3: Advanced AI (demand forecasting, supplier scoring, what-if simulator).
- Phase 4: Multi-company, multi-branch, multi-currency, workflow builder, data warehouse.

Out of scope for the first version

- Real money movement through a payment provider. The first version uses sandbox simulation and external-transfer recording; real integration only after confirming the provider officially supports it and checking legal/CBE requirements.
- Autonomous AI actions: the AI suggests, but never pays, changes a budget or approves a transaction by itself.

1.5 Target Users

Company Owner/CEO, Finance Manager/CFO, Accountant, Department Manager, Procurement Officer, Inventory/Warehouse Officer, Payroll/HR Officer, Production Manager, Sales Officer, Employee/Requester, Auditor (read-only), and External Supplier.

1.6 Proposed Technology Stack

Layer	Technology
Frontend	React, TypeScript, Vite, Tailwind CSS, chart library
Backend	ASP.NET Core Web API, C#, Entity Framework Core
Database	SQL Server
AI Service	Python, FastAPI, pandas, scikit-learn, LLM integration
Security & Infrastructure	ASP.NET Core Identity, JWT/Cookies, Role/Policy authorization, Docker, CI/CD, logging and monitoring
Architecture	Modular Monolith (clear modules: Finance, Procurement, Inventory, and so on)

1.7 Assumptions

- The system is designed as a general, customizable system for one company with several branches and departments (final decision to be confirmed in the SRD).
- Performance, availability and user-volume numbers are fixed in the SRD after agreeing on system size.
- Any technology not listed above is added only when there is a clear need.

2. Task Assignment & Roles

2.1 Team Roles

#	Member	Role	Main Responsibilities
1	Menna Atef	Team Lead, Backend & AI Developer	Project coordination; GitHub management; Finance Core; accounting ledger; approval workflow; AI integration and forecasting
2	Nourhan Fawzy	Frontend & Backend Developer	React UI; dashboards; backend APIs; business logic; database integration; responsive design
3	Nour Hany	Backend & AI Developer	Backend APIs; financial data processing; AI service integration; anomaly detection; forecasting
4	Weam Emad	Frontend & Backend Developer	React UI; responsive layouts; backend APIs; procurement; inventory; payments integration
5	Omar Mohamed	Backend & DevOps Engineer	Backend APIs; business logic; Docker; CI/CD pipelines; deployment; environment configuration; automated testing

2.2 Shared Responsibilities (All Members)

- Collaborative development across frontend, backend, AI, and DevOps tasks.
- Joint preparation of all project documentation and DEPI deliverables.
- Documentation of requirements, diagrams, APIs, design decisions, and implementation details.
- Code reviews through pull requests, with at least one reviewer per PR.
- Participation in testing, bug fixing, and quality assurance.
- Participation in the final presentation and project demo.
- Menna Atef, as Team Lead, reviews all project documentation, deliverables, and overall team progress.

2.3 Documentation and Deliverables

All team members collaborate on preparing and completing the DEPI project deliverables, while Menna Atef, as Team Lead, reviews the work to ensure quality, consistency, and compliance with project requirements.

2.4 Ways of Working

- Use Jira to organize project tasks, track progress, manage priorities, and monitor deadlines.
- Use a shared GitHub repository with feature branches, meaningful commit messages, and pull request descriptions.
- Collaborate on all project modules and documentation rather than assigning exclusive ownership of entire modules to individual members.
- Create Jira tasks with clear descriptions, priorities, acceptance criteria, and due dates.
- Hold weekly team meetings and quick check-ins before each DEPI deadline.
- Review and agree on changes to requirements, architecture, and database design as a team.
- Keep Jira updated to reflect task status, blockers, and completed work.
- Menna Atef reviews overall progress and deliverables to ensure consistency, quality, and alignment with project requirements.

3. Risk Assessment & Mitigation Plan

Likelihood and impact: **H** = High, **M** = Medium, **L** = Low.

ID	Risk	Likelihood	Impact	Mitigation	Owner
R1	Scope is too large for the time available (the Blueprint covers many modules)	H	H	Build only Phase 1 first; keep Phase 2–4 as future work; freeze scope after the requirements deadline	Member 1
R2	Wrong financial numbers (budget, committed, spent, available)	M	H	Atomic transactions, clear definitions of each amount, unit tests on budget calculations, review by a second member	Member 1
R3	Duplicate payments or payment confirmed twice	M	H	Idempotency key per payment, unique reference constraint, payment status machine (Pending, Processing, Paid, Failed, Refunded), tests for repeated confirmations	Member 2
R4	Concurrent updates exceed the budget (race condition)	M	H	Database transactions and concurrency control (row versioning), concurrent-update tests	Member 1
R5	Permission leaks (a user sees data they should not, such as salaries)	M	H	Backend checks on every request, RBAC plus data scope, automated permission tests per role	Member 2
R6	Real payment integration is not available or not allowed	H	M	Use sandbox simulation and external-transfer recording in the first version; verify provider capabilities and legal requirements before any real integration	Member 2
R7	AI results are weak or not explainable (limited or poor data)	M	M	Generate realistic seed/synthetic data, start with rule-based detection, show data source and date with each result, keep human approval	Member 4
R8	AI assistant exposes data beyond the user's permissions	L	H	Assistant uses only permission-checked read tools; no direct database access; log assistant queries	Member 4
R9	Integration problems between the .NET backend, the Python AI service and the React frontend	M	M	Define API contracts early (OpenAPI), mock services, integration tests, Docker Compose for the full stack	Member 5
R10	Team member unavailable or uneven workload	M	M	Pair on critical modules, keep shared documentation, redistribute tasks at weekly sync	Member 1
R11	Missing DEPI deadlines	M	H	Work backwards from each deadline, weekly progress review, submit early drafts to GitHub	Member 1
R12	Data loss or broken environment during development	L	M	GitHub for code, database scripts and seed data under version control, regular backups, restore test	Member 5
R13	Low usability (complex financial screens)	M	M	Early wireframes, short usability tests with classmates, clear error messages, role-specific dashboards	Member 3

4. KPIs (Key Performance Indicators)

The targets below are initial proposed values and may be adjusted based on the project's final requirements and specifications in the SRD.

4.1 Product and Functional KPIs

KPI	Target	Measurement
End-to-end business workflow success	100%	Demo checklist and tests
Phase 1 modules delivered	100%	SRD feature checklist
Requirements covered by test cases	At least 90%	Traceability matrix
Financial accuracy (budget and ledger)	100%	Reconciliation tests

4.2 Technical KPIs

KPI	Target	Measurement
API response time	Under 2 seconds	API tests and logs
Dashboard load time	Under 3 seconds	Browser performance tools
Automated test pass rate	At least 95%	CI pipeline results
Unit test coverage (financial and permission logic)	At least 70%	Coverage report
Critical bugs at final delivery	0	Bug tracker

4.3 Security and AI KPIs

KPI	Target	Measurement
Unauthorized access prevention	100% in permission tests	Role-based tests
Sensitive actions recorded in audit log	100%	Audit log review
AI respects user permissions	100% in test cases	Permission tests per role
Cash-flow forecasting	Evaluated against a baseline	MAPE or similar metric
Anomaly detection	Detects test anomalies	Precision and recall

4.4 Project Management and Adoption KPIs

KPI	Target	Measurement
DEPI deliverables submitted on time	100%	GitHub records
Milestones completed on schedule	At least 90%	Jira task board
Pull requests reviewed before merge	100%	GitHub PR history
User satisfaction in usability tests	At least 4 out of 5	User questionnaire
Task success rate in usability tests	At least 90%	Observed test sessions